

VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

School of Computer Science and Engineering (SCOPE)

CODEFLOW

A Domain-Specific Language and Compiler for Workflow Automation

Phase 1 Project Proposal and Design Document

(Problem Definition and System Design)

Course Title	Compiler Design Laboratory
Course Code	BCSE307P
Project Type	Individual Project
Student Name	Daksh Arora
Register Number	24BCI0259
Program / Branch	Computer Science and Engineering (Information Security)

Vellore Institute of Technology, Vellore
Academic Year 2024–2028

Table of Contents

- 1. Project Title..... 3
- 2. Abstract.....4
- 3. Problem Statement.....5
- 4. Motivation..... 6
- 5. Objectives.....7
- 6. Scope of the Project.....8
- 7. Background Study..... 10
- 8. Compiler Design Concepts Involved..... 11
- 9. CodeFlow Language Specification..... 13
- 10. Requirement Analysis..... 18
- 11. Proposed Methodology..... 20
- 12. System Architecture.....21
- 13. Module Description.....23
- 14. Technology Stack.....24
- 15. Initial Prototype.....25
- 16. Innovation and Technical Contribution.....26
- 17. Expected Outcomes.....27

1. Project Title

Title: CodeFlow — A Domain-Specific Language and Compiler for Workflow Automation

Course: Compiler Design Laboratory (BCSE307P)

Phase: Phase 1 — Problem Definition and System Design

Project Type: Individual Project

2. Abstract

Workflow logic — conditional rules that react to sensor readings or incoming data and trigger actions such as alerts, logs, or automated tasks — is commonly expressed either informally inside general-purpose programming languages or through rigid graphical rule-builders. Neither option offers a syntax that is simultaneously readable by a non-programmer domain expert and precise enough to be compiled and analyzed formally. This project proposes CodeFlow, a small, textual Domain-Specific Language (DSL) purpose-built for expressing conditional workflow logic, together with a complete compiler and execution pipeline for the language. The pipeline follows the classical compiler construction model: a lexical analyzer converts CodeFlow source text into a token stream; a recursive-descent parser validates the token stream against a formally specified grammar and constructs an Abstract Syntax Tree (AST); a semantic analyzer checks type correctness and variable declarations using a symbol table; an intermediate-code generator lowers the validated AST into Three-Address Code (TAC); a code optimizer applies feasible optimizations such as constant folding and dead-code elimination; and an execution engine interprets the optimized intermediate representation to produce a final result. A supporting visualization interface allows each intermediate artifact — tokens, AST, symbol table, IR, and execution trace — to be inspected directly, and a dedicated error-handling subsystem reports lexical, syntax, semantic, and runtime errors with meaningful diagnostics. This document constitutes the Phase 1 deliverable of the project: it defines the problem, motivation, objectives, and scope; documents background study in compiler theory and DSL design; specifies the CodeFlow language formally, including its grammar; analyzes functional and non-functional requirements; and presents the proposed system architecture, module design, and technology stack. It also documents the current status of the initial prototype, which implements the lexical analyzer and a basic recursive-descent parser capable of producing a preliminary AST and reporting elementary syntax errors. Semantic analysis, intermediate-code generation, optimization, and execution are scoped for Phases 2 and 3 respectively, as detailed in the three-phase roadmap (Section 6.3).

3. Problem Statement

Automated conditional logic of the form “when a condition holds on a sensor reading or an input value, perform an action” recurs across a wide range of small-scale automation scenarios — environmental monitoring, order-processing checks, notification systems, and simple IoT control logic. In current practice, such logic is typically implemented in one of two ways. The first is to write it directly in a general-purpose language (Python, JavaScript, Java), which is expressive but requires the workflow author to understand full language semantics, deal with unrelated language features, and manually re-implement structural patterns (condition evaluation, branching, action dispatch) that are common to nearly every workflow. The second is to configure the logic through a graphical, GUI-driven workflow builder, which is easier for non-programmers but offers little precision, is difficult to version, test, or statically analyze, and cannot be compiled or optimized as a formal language.

There is, therefore, a gap for a small, textual, domain-specific language that (a) is restricted enough in scope to be learned quickly and reasoned about formally, (b) directly expresses the workflow vocabulary — sensors, inputs, conditions, actions, alerts, and logs — as first-class syntax, and (c) is processed by a proper compiler pipeline rather than an ad-hoc interpreter, so that the language benefits from systematic lexical analysis, syntax and semantic validation, an explicit intermediate representation, and defined error handling. The core problem this project addresses is the design of such a language, CodeFlow, and the design and implementation of a compiler and execution engine for it, applying standard compiler construction techniques end-to-end rather than treating the workflow domain as a generic scripting or web-application problem.

4. Motivation

The motivation for this project is primarily to gain hands-on, end-to-end experience in compiler construction by applying it to a concrete, self-contained domain rather than to an already well-understood toy language. Building a compiler for a language with a genuine (if narrow) application domain forces engagement with real design questions that a purely academic grammar exercise does not raise: how to design keywords and syntax that read naturally to a domain user, how to decide what belongs in the grammar versus what is deferred to the runtime, how to design a symbol table and type system for a domain with only a few types, and how to represent domain concepts such as conditional branching and action dispatch in an intermediate representation.

A secondary motivation is to consolidate the theoretical material of the Compiler Design Laboratory course — finite automata and regular expressions for lexical analysis, context-free grammars and recursive-descent/LL parsing, AST construction, scope and type checking, three-address code generation, basic optimization passes, and tree-walking interpretation — into a single working system, built and understood individually from source-level input to executed output, including a visualization layer that makes each internal compiler artifact observable and easier to reason about during development and demonstration.

5. Objectives

The specific objectives of the project are as follows:

1. To design a small, well-defined Domain-Specific Language, CodeFlow, with a formal lexical and grammatical specification suitable for expressing sensor- and input-driven conditional workflow logic.
2. To implement a lexical analyzer that tokenizes CodeFlow source programs and reports lexical errors (illegal characters, malformed literals) with source position information.
3. To implement a recursive-descent parser that validates the token stream against the CodeFlow grammar and constructs an Abstract Syntax Tree, reporting syntax errors with recovery information where feasible.
4. To implement a semantic analyzer that performs declaration, scope, and type checking over the AST using a symbol table, detecting semantic errors such as undeclared identifiers and type mismatches.
5. To design and generate an intermediate representation (Three-Address Code) for validated CodeFlow programs that captures condition evaluation, branching, and action dispatch.
6. To apply feasible code optimization techniques — constant folding, dead-code elimination, and unreachable-code elimination — to the generated intermediate representation.
7. To implement an execution engine that interprets the optimized intermediate representation and produces an observable execution result.
8. To provide a visualization interface that exposes each compiler-internal artifact (tokens, AST, symbol table, IR, optimized IR, and errors) for inspection and debugging.
9. To design and implement a consistent, layered error-handling mechanism spanning lexical, syntax, semantic, and runtime stages.

6. Scope of the Project

6.1 In-Scope

- Design of the CodeFlow DSL: keywords, data types, operators, statement forms, and a formal grammar (BNF/EBNF).
- A hand-written lexical analyzer producing a classified token stream with source-position tracking.
- A hand-written recursive-descent parser producing an Abstract Syntax Tree, with syntax error detection.
- A semantic analysis pass with a symbol table supporting declaration, scope, and type checking (Phase 2).
- Generation of Three-Address Code as the intermediate representation for validated programs (Phase 2).
- A small set of feasible optimizations over the intermediate representation (Phase 3).
- A tree-walking / IR-walking execution engine that runs a compiled CodeFlow program and reports the result (Phase 3).
- A lightweight visualization interface for inspecting tokens, AST, symbol table, IR, optimized IR, and errors.
- Layered error handling covering lexical, syntax, semantic, and applicable runtime errors.

6.2 Out-of-Scope

- General-purpose programming constructs not required by the workflow domain: user-defined functions, loops, recursion, complex data structures (arrays, objects, classes), and modules/imports.
- Concurrency, distributed execution, networking, or integration with real physical sensors or external hardware.
- Persistent storage, multi-user accounts, authentication, or any enterprise workflow-management features (dashboards, scheduling servers, role-based access).
- Machine-code or native binary code generation; the project targets an interpreted intermediate representation only.
- Advanced compiler optimizations such as register allocation, instruction scheduling, or inter-procedural analysis, which are not applicable to an interpreted, function-free DSL of this scope.
- Production-grade performance engineering, large-scale testing infrastructure, or formal verification of the compiler itself.

6.3 Three-Phase Roadmap

Phase	Focus	Key Deliverables
Phase 1	Problem Definition & System Design	Problem statement, objectives, CodeFlow language specification, formal grammar, system architecture, technology stack, and an initial lexer/parser/AST prototype.
Phase 2	Core Compiler Construction	Complete lexer and parser, full AST, symbol table, semantic analysis (type and scope checking), IR/TAC generation, and layered error handling.
Phase 3	Optimization, Execution & Integration	Code optimization passes, execution engine, visualization/integration of all stages, comprehensive testing, and final documentation.

7. Background Study

Compiler construction is a mature and well-documented area of computer science, structured classically into a front end (lexical analysis, syntax analysis, semantic analysis) and a back end (intermediate code generation, optimization, and code generation or execution). The standard reference treatment of this pipeline, including finite-automaton-based lexical analysis, LL/LR parsing theory, syntax-directed translation, symbol table design, and intermediate representations such as three-address code, is given in Aho, Lam, Sethi, and Ullman's *Compilers: Principles, Techniques, and Tools*. A complementary, implementation-oriented treatment — closer in spirit to this project, which builds a tree-walking interpreter rather than a native code generator — is given in Nystrom's *Crafting Interpreters*, which works through lexer, recursive-descent parser, AST, and tree-walking interpreter construction for a small dynamically-typed language.

A Domain-Specific Language differs from a general-purpose language primarily in the deliberate narrowing of its vocabulary and constructs to a single problem domain, trading generality for readability, safety, and ease of analysis within that domain. Existing rule-based and event-condition-action systems (spreadsheet formulas, IFTTT-style automation rules, Prometheus/Grafana alerting rules) illustrate the same underlying pattern that CodeFlow formalizes as a compiled language: a declared set of monitored quantities, a condition, and one or more resulting actions. The distinguishing characteristic of this project relative to such systems is that CodeFlow is processed by an explicit compiler pipeline — with tokenization, a formal grammar, an AST, static semantic checks, and an intermediate representation — rather than being interpreted directly off a configuration file or a spreadsheet-style expression evaluator.

On the implementation-technology side, background study also covered parser construction strategies. Parser-generator tools such as Lex/Flex (lexer generators) and Yacc/Bison/ANTLR (parser generators) automate lexer and parser construction from a grammar specification, and are recommended in the course manual as acceptable approaches. For an individual project with a grammar of CodeFlow's modest size, a hand-written recursive-descent parser was judged preferable to a generated parser because it keeps the full pipeline (from character stream to AST) transparent and directly modifiable in a single codebase, which aids both learning outcomes and error-message quality — a trade-off discussed further in Section 14 (Technology Stack).

8. Compiler Design Concepts Involved

This section maps the classical stages of compiler construction taught in the Compiler Design Laboratory course to their specific realization within the CodeFlow compiler.

8.1 Lexical Analysis

The lexer scans raw CodeFlow source text and groups characters into tokens belonging to fixed categories (keywords, identifiers, literals, operators, delimiters), discarding whitespace and comments. It is designed conceptually as a deterministic finite automaton over character classes (letters, digits, quote characters, operator symbols), implemented as a hand-written character-scanning routine. Lexical errors — illegal characters, unterminated string literals, malformed numeric literals — are detected at this stage and reported with line and column position.

8.2 Syntax Analysis / Parsing

The parser consumes the token stream produced by the lexer and validates it against the CodeFlow context-free grammar (Section 9.12), using a hand-written recursive-descent strategy with one token of lookahead. Each grammar production is realized as a parsing function; a successful parse constructs an Abstract Syntax Tree node, while a grammar violation raises a syntax error identifying the expected and found tokens.

8.3 Abstract Syntax Tree (AST)

The AST is the parser's structured output: an intermediate, tree-shaped representation of a CodeFlow program that discards concrete syntax (braces, keywords used purely as delimiters) while retaining program structure and semantics-relevant detail (declarations, conditions, statement bodies). The AST is the shared data structure over which semantic analysis, IR generation, and the visualization layer subsequently operate, and it is designed to be traversable via a standard visitor/recursive-traversal pattern.

8.4 Semantic Analysis

Semantic analysis walks the AST after a successful parse to check properties that a context-free grammar cannot express: that every identifier used (in a condition, an action, or a log/alert statement) has been declared as a sensor or input within its enclosing workflow, that operators are applied to operands of compatible types (for example, that a relational operator is not applied to a boolean and a string), and that literals conform to their declared types. This stage is planned for Phase 2.

8.5 Symbol Table

The symbol table records, for each declared sensor or input, its name, kind (sensor or input), declared type, and enclosing workflow scope. It supports insertion at declaration, lookup during

semantic analysis and IR generation, and scope resolution, since each workflow block defines its own declaration scope, as illustrated in Section 9 and detailed in Section 8.5.

8.6 Intermediate Code Generation

Validated AST nodes are lowered into Three-Address Code (TAC), a linear sequence of simple instructions (at most one operator per instruction, with named temporaries) that represents condition evaluation, conditional branching between when/otherwise blocks, and action/alert/log dispatch. TAC is chosen because it is close enough to the AST to generate directly by tree traversal, yet linear and simple enough to optimize and interpret directly, without requiring a target machine instruction set. This stage is planned for Phase 2.

8.7 Code Optimization

Because CodeFlow programs are typically small, dominated by condition-checking rather than heavy computation, optimization focuses on a small set of feasible, clearly beneficial transformations over the TAC: constant folding (evaluating constant sub-expressions such as comparisons between two literals at compile time), dead-code elimination (removing computations whose results are never used), and unreachable-code elimination (removing code following an unconditional branch or a statically-determined-false condition). These are planned for Phase 3 and are illustrated conceptually in Section 8.7 and Section 12.

8.8 Execution

The execution engine interprets the (optimized) TAC sequence directly — evaluating expressions, following conditional branches, and dispatching action/alert/log statements to a simple runtime that records or prints their effect, since CodeFlow has no notion of physical actuation within the scope of this project. This stage is planned for Phase 3.

8.9 Error Handling

Error handling is treated as a cross-cutting concern rather than a single stage: the lexer reports lexical errors, the parser reports syntax errors, the semantic analyzer reports semantic errors (undeclared identifiers, type mismatches), and the execution engine reports applicable runtime errors (such as division by zero within a condition expression). All error categories are surfaced through a common error-reporting format (stage, message, source location) so that the visualization layer can display them uniformly, as shown in Figure 1.

9. CodeFlow Language Specification

9.1 Purpose of the Language

CodeFlow is a small, statically-typed, textual Domain-Specific Language for expressing conditional workflow logic: declaring named sensors and inputs, testing conditions over their values, and triggering actions, alerts, or log messages in response. It is deliberately restricted to this domain — it is not a general-purpose programming language, and it intentionally omits functions, loops, recursion, and complex data structures so that its grammar and semantics remain small enough to specify and compile exhaustively within the scope of an individual compiler-construction project.

9.2 Keywords

Keyword	Meaning
workflow	Declares a named workflow block — the top-level unit of a CodeFlow program.
sensor	Declares a sensor-backed variable with a type, populated externally at runtime.
input	Declares an input variable with a type, supplied to the workflow at runtime.
when	Introduces a conditional block, executed if its condition evaluates to true.
otherwise	Introduces a fallback block, executed if the preceding when condition is false.
action	Invokes a named action (a side-effecting operation) with an optional argument list.
alert	Emits an alert message (a string literal) from within a block.
log	Emits a diagnostic log message (a string literal) from within a block.
number, string, boolean	The three built-in primitive type names usable in declarations.
AND, OR, NOT	Logical operators usable within condition expressions.
true, false	Boolean literal keywords.

9.3 Identifiers

An identifier names a workflow-scoped sensor, input, or action. It must begin with a letter (a–z, A–Z) or underscore, followed by zero or more letters, digits, or underscores, and must not coincide with a reserved keyword.

```
identifier ::= (letter | "_") (letter | digit | "_")*
```

9.4 Literals

Literal Kind	Description	Example
Number literal	An integer or decimal numeric constant.	35, 10000, 2.5
String literal	A sequence of characters enclosed in double quotes.	"High temperature"
Boolean literal	One of the two reserved keywords true or false.	true

9.5 Data Types

CodeFlow supports exactly three primitive data types, matching the values a sensor or input can carry in the workflow automation domain:

- number — numeric values (integers and decimals), used for sensor readings and numeric inputs.
- string — textual values, used chiefly in alert and log message literals.
- boolean — logical true/false values, used for condition results and boolean sensors/inputs.

9.6 Operators

Category	Operators	Operand Types	Result Type
Arithmetic	+ - * /	number, number	number
Relational	> < >= <= == !=	number, number (== / != also on string, boolean)	boolean
Logical	AND OR NOT	boolean (operands)	boolean

9.7 Delimiters and Separators

Symbol	Purpose
{ }	Delimit a workflow body or a when/otherwise block.
()	Delimit an action's argument list or a parenthesized sub-expression.
:	Separates a declared identifier from its type in a sensor/input declaration.
,	Separates arguments in an action's argument list.
" "	Delimit a string literal.

9.8 Statements

A CodeFlow workflow body is composed of declaration statements and workflow statements:

- Declaration statements — sensor and input declarations, which bind an identifier to a type within the enclosing workflow's scope.
- Conditional statements — a when block (with an associated condition) optionally paired with a following otherwise block.
- Action statements — an action invocation, given as an identifier followed by a parenthesized, possibly empty argument list.
- Alert and log statements — a keyword (alert or log) followed by a string-literal message.

9.9 Workflow Structure

Every CodeFlow program is a sequence of one or more top-level workflow declarations. Each workflow has a name (a string literal), a body containing zero or more declarations, followed by zero or more workflow statements (conditionals, actions, alerts, logs) which may appear directly in the workflow body or nested inside when/otherwise blocks.

9.10 Example Program 1 — Sensor-Driven Workflow

```
workflow "CoolingSystem" {
    sensor temperature : number

    when temperature > 35 {
        alert "High temperature"
        action start_fan()
    }

    otherwise {
        log "Temperature normal"
    }
}
```

9.11 Example Program 2 — Input-Driven Workflow

```
workflow "OrderProcessing" {
    input orderAmount : number

    when orderAmount > 5000 {
        action fraud_check()
    }
}
```

```

    when orderAmount > 10000 {
        action manager_approval()
    }

    action process_payment()
}

```

9.12 Formal Grammar (EBNF)

The following grammar is the working Phase 1 specification of CodeFlow syntax, consistent with the example programs above and with the parser design in Section 13. Terminals are quoted; “{ }” denotes zero-or-more repetition and “[]” denotes an optional element, in standard EBNF notation.

```

program      ::= { workflowDecl }
workflowDecl ::= "workflow" STRING "{" { declaration } { statement } "}"
declaration  ::= ("sensor" | "input") IDENT ":" type
type         ::= "number" | "string" | "boolean"
statement    ::= whenStmt
               | otherwiseStmt
               | actionStmt
               | alertStmt
               | logStmt
whenStmt     ::= "when" expression block
otherwiseStmt ::= "otherwise" block
block        ::= "{" { statement } "}"
actionStmt   ::= "action" IDENT "(" [ argList ] ")"
alertStmt    ::= "alert" STRING
logStmt      ::= "log" STRING
argList      ::= expression { "," expression }
expression   ::= orExpr
orExpr       ::= andExpr { "OR" andExpr }

```

```

andExpr      ::= notExpr { "AND" notExpr }
notExpr      ::= [ "NOT" ] relExpr
relExpr      ::= addExpr [ relOp addExpr ]
relOp        ::= ">" | "<" | ">=" | "<=" | "==" | "!="
addExpr      ::= mulExpr { ("+" | "-") mulExpr }
mulExpr      ::= unaryExpr { ("*" | "/") unaryExpr }
unaryExpr    ::= [ "-" ] primary
primary      ::= NUMBER | STRING | BOOLEAN | IDENT | "(" expression ")"

IDENT        ::= (letter | "_") (letter | digit | "_")*
NUMBER       ::= digit { digit } [ "." digit { digit } ]
STRING       ::= "'" { any character except "'" } "'"
BOOLEAN      ::= "true" | "false"

```

9.13 Operator Precedence and Associativity

Operator precedence follows the standard convention of binding logical disjunction most loosely and unary negation most tightly, which is reflected directly in the layering of the grammar productions above (each precedence level is a separate non-terminal). All binary operators are left-associative.

Precedence	Operator(s)	Associativity	Description
1 (lowest)	OR	Left	Logical disjunction
2	AND	Left	Logical conjunction
3	NOT	Right (unary)	Logical negation
4	> < >= <= == !=	Left	Relational comparison
5	+ -	Left	Additive arithmetic
6	* /	Left	Multiplicative arithmetic
7 (highest)	- (unary)	Right (unary)	Unary numeric negation

10. Requirement Analysis

10.1 Functional Requirements

ID	Requirement
FR-1	The system shall accept a CodeFlow source program as text input.
FR-2	The system shall tokenize the source program, classifying each token by category and recording its source position.
FR-3	The system shall parse the token stream against the CodeFlow grammar and construct an Abstract Syntax Tree.
FR-4	The system shall perform semantic validation over the AST, including scope and type checking, using a symbol table.
FR-5	The system shall maintain a symbol table recording each declared identifier's name, kind, type, and scope.
FR-6	The system shall generate an intermediate representation (Three-Address Code) for semantically valid programs.
FR-7	The system shall apply feasible optimizations to the generated intermediate representation.
FR-8	The system shall execute valid, optimized workflows and report the resulting actions, alerts, and logs.
FR-9	The system shall detect and report lexical, syntax, semantic, and applicable runtime errors with source location information.
FR-10	The system shall display, on request, the output of each compiler stage (tokens, AST, symbol table, IR, optimized IR, and errors).

10.2 Non-Functional Requirements

Quality Attribute	Description
Correctness	The compiler must faithfully implement the specified CodeFlow grammar and semantics at every stage.
Modularity	Lexer, parser, semantic analyzer, IR generator, optimizer, and execution engine shall be independently testable modules with clearly defined interfaces.
Maintainability	Code shall be organized and documented so that a stage can be modified or extended without requiring changes to unrelated stages.
Explainability	Every intermediate artifact shall be inspectable through the visualization layer, in support of teaching and demonstration goals.

Quality Attribute	Description
Usability	Error messages shall be specific (stage, expected vs. found, source location) rather than generic failures.
Extensibility	The design shall allow later addition of further statement forms, types, or optimizations without a full rewrite.
Reasonable Efficiency	Compilation and execution of representative CodeFlow programs shall complete interactively (well under one second) given the small expected program sizes.
Clear Error Reporting	Each error category (lexical/syntax/semantic/runtime) shall be reported through one consistent, uniformly formatted channel.

10.3 Input, Output, and Software Requirements

Category	Requirement
Input	One or more syntactically well-formed or intentionally malformed CodeFlow source files (.cflow), authored as plain text.
Output	Token list, AST representation, symbol table contents, generated and optimized IR, execution trace/result, and any diagnostic errors, in a human-readable form.
Software / Runtime	Node.js runtime and TypeScript compiler for building and executing the CodeFlow compiler; a modern web browser for the visualization interface.
Development Tools	Visual Studio Code as the IDE; Git and GitHub for version control and submission history.
Hardware	A standard personal computer; no specialized or embedded hardware is required, since sensor values are supplied as declared inputs rather than read from physical devices.

11. Proposed Methodology

The project follows the standard compiler-construction methodology, implementing each stage as a discrete module that consumes the output of the previous stage and produces a well-defined artifact for the next, as illustrated in Figure 1 (Section 12).

1. **Source Acquisition:** A CodeFlow source file is read as plain text; source text and line/column bookkeeping are retained for later error reporting.
2. **Lexical Analysis:** The lexer scans the source character-by-character, recognizing keywords, identifiers, literals, operators, and delimiters, and produces an ordered token stream. Illegal characters or malformed literals are reported as lexical errors at this stage.
3. **Syntax Analysis:** The recursive-descent parser consumes the token stream, matching it against the CodeFlow grammar (Section 9.12) one production at a time, and builds an Abstract Syntax Tree. Grammar violations are reported as syntax errors, naming the expected and encountered tokens.
4. **Semantic Analysis:** The semantic analyzer traverses the AST, inserting each sensor/input declaration into a scoped symbol table and checking that every identifier reference resolves to a prior declaration of a compatible type, and that operators are applied to type-compatible operands. (Phase 2.)
5. **Intermediate Code Generation:** The IR generator performs a second AST traversal, emitting Three-Address Code instructions for condition evaluation, conditional branching between when/otherwise blocks, and action/alert/log dispatch. (Phase 2.)
6. **Code Optimization:** The optimizer processes the generated TAC to fold constant expressions, remove dead assignments, and eliminate unreachable code following a statically-resolved branch. (Phase 3.)
7. **Execution:** The execution engine interprets the optimized TAC sequentially, evaluating expressions against supplied sensor/input values and invoking the corresponding action/alert/log effects, producing the final execution result. (Phase 3.)
8. **Error Handling (cross-cutting):** At every stage above, detected errors are captured in a common error-record format and surfaced through the visualization layer rather than halting the whole pipeline where recovery is feasible.
9. **Visualization:** Each stage additionally exposes its output artifact (tokens, AST, symbol table, IR, optimized IR, result, errors) to a lightweight interface for inspection, aiding both development and Review demonstration.

12. System Architecture

Figure 1 shows the end-to-end system architecture of the CodeFlow compiler and execution environment. Source text flows left to right through the lexer, parser, semantic analyzer, IR generator, optimizer, and execution engine, each producing the artifact consumed by the next stage. The symbol table sits alongside the semantic analyzer, which populates it during declaration processing and consults it during reference resolution; later stages (IR generation, execution) also consult it for type and identifier information. Two cross-cutting subsystems span the whole pipeline: an error handler, attached to every stage capable of producing a diagnostic, and a visualization layer, attached to every stage that produces an inspectable artifact.

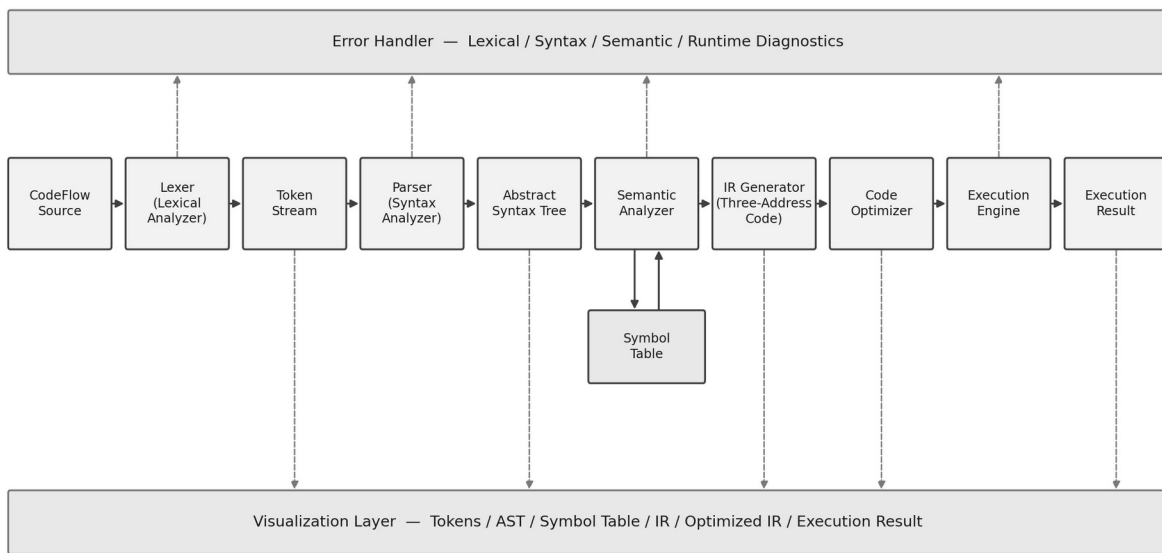


Figure 1: System Architecture of the CodeFlow Compiler and Execution Environment

The architecture is deliberately linear and single-pass in the forward direction (no stage re-invokes an earlier stage), which keeps the design tractable for an individual project while still exercising every classical compiler phase. The only bidirectional interaction in the diagram is between the semantic analyzer and the symbol table, which is populated and queried repeatedly as declarations and references are encountered during a single AST traversal.

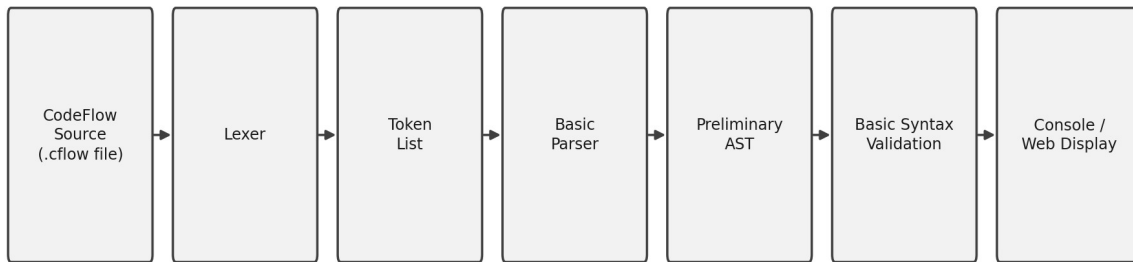


Figure 2: Data Flow of the Phase 1 Initial Prototype

Figure 2 shows the reduced pipeline actually implemented in the Phase 1 prototype: source text is tokenized, parsed by a basic recursive-descent parser into a preliminary AST, subjected to basic syntax validation, and displayed. Semantic analysis, IR generation, optimization, and execution (shown in Figure 1) are not yet implemented and are scheduled for Phases 2 and 3 as described in Section 6.3.

13. Module Description

The system is decomposed into eight modules, each with a single, clearly bounded responsibility, matching the pipeline stages introduced in Sections 11 and 12.

Module	Responsibility	Input	Output
Lexer	Tokenizes raw source text; detects lexical errors.	CodeFlow source text	Token stream
Parser	Validates syntax against the grammar; builds the AST; detects syntax errors.	Token stream	Abstract Syntax Tree
Semantic Analyzer	Performs scope and type checking over the AST; populates/queries the symbol table.	AST	Annotated AST; semantic errors
Symbol Table	Stores identifier name, kind, type, and scope; supports insert, lookup, and scope resolution.	Declarations from semantic analyzer	Identifier records on lookup
IR Generator	Lowers the annotated AST into Three-Address Code.	Annotated AST	TAC instruction sequence
Optimizer	Applies constant folding, dead-code, and unreachable-code elimination to the TAC.	TAC instruction sequence	Optimized TAC instruction sequence
Execution Engine	Interprets the optimized TAC; dispatches action/alert/log effects; detects runtime errors.	Optimized TAC; sensor/input values	Execution result / trace
Visualization Layer	Renders tokens, AST, symbol table, IR, optimized IR, and errors for inspection.	Artifacts from all other modules	Human-readable / graphical display

14. Technology Stack

The technology choices below follow the tools recommended in the course manual (C, C++, Python, Java, JavaScript, TypeScript for implementation; Lex/Flex, Yacc/Bison/ANTLR as optional parser-generation aids), selected and justified for an individual project of CodeFlow's scope.

Layer	Technology	Justification
Compiler implementation language	TypeScript	Static typing aids correctness when modeling tokens, AST node types, and IR instructions as discriminated unions; compiles to JavaScript for both a command-line tool and a browser-based visualization front end from one codebase.
Parsing approach	Hand-written recursive-descent parser	CodeFlow's grammar (Section 9.12) is small and LL(1)-friendly; a hand-written parser keeps the whole front end transparent and directly debuggable, and produces more specific syntax-error messages than a generated parser, at the cost of more manual grammar-to-code translation than a tool such as ANTLR would require.
Execution strategy	TypeScript/JavaScript tree- or IR-walking interpreter	Avoids native code generation, which is unnecessary for CodeFlow's scope and interpreted-execution goals; runs directly on Node.js without an external runtime dependency.
Visualization interface	HTML, CSS, and JavaScript (lightweight, no framework dependency assumed)	Allows the compiler-internal artifacts (tokens, AST, symbol table, IR) to be rendered as simple structured views without introducing a heavyweight front-end framework.
Development environment	Visual Studio Code	Integrated TypeScript tooling, debugging, and extension support for an individual development workflow.
Version control	Git and GitHub	Tracks incremental development across phases and provides a submittable, reviewable history of the implementation.

Parser-generator tools (Lex/Flex, Yacc/Bison, ANTLR) were considered and are acknowledged as valid alternative approaches per the course manual, but were not selected for the core implementation, since a hand-written lexer and parser better serve the project's learning objective of implementing every pipeline stage directly rather than delegating lexer/parser construction to a generator.

15. Initial Prototype

A minimal Phase 1 prototype has been designed around the reduced pipeline shown in Figure 2 (Section 12): CodeFlow source text is read, tokenized by the lexer, and passed to a basic recursive-descent parser that builds a preliminary AST covering workflow declarations, sensor/input declarations, when/otherwise blocks, and action/alert/log statements, together with basic syntax validation (matching braces and parentheses, expected-token checks at each grammar production). The prototype demonstrates, for a given input program:

- The generated token list, with each token's category, lexeme, and source position.
- The preliminary AST structure resulting from a successful parse of a well-formed program such as the two examples in Section 9.10–9.11.
- Basic syntax-error detection and reporting for intentionally malformed input (for example, a missing closing brace or an unrecognized statement keyword).

Consistent with the phase-wise scope defined in Section 6.3, the Phase 1 prototype does not yet perform semantic analysis, symbol-table-based type checking, intermediate code generation, optimization, or execution; these are planned deliverables of Phases 2 and 3 respectively, described in Section 6.3. No performance figures, test-suite results, or execution outputs are reported at this stage, since these components do not yet exist.

16. Innovation and Technical Contribution

The project's contribution is modest in absolute scale, appropriate to an individual compiler-laboratory project, but combines several elements that are not typically found together in a course exercise:

- A purpose-built DSL, rather than a generic toy grammar, whose keywords and structure (sensor, input, when/otherwise, action, alert, log) are chosen specifically to express the workflow-automation domain directly, rather than being retrofitted onto an existing general-purpose syntax.
- A compiler pipeline designed specifically for this DSL's structure — in particular, an intermediate representation and execution model built around condition evaluation and action dispatch rather than around arithmetic expression evaluation alone, as is typical of introductory compiler exercises.
- A visualization layer that exposes every internal compiler artifact (tokens, AST, symbol table, IR, optimized IR) rather than only the final output, supporting both development debugging and clearer demonstration of compiler internals during evaluation.
- Domain-specific semantic validation — for example, restricting comparison operators to type-compatible operands drawn from the three domain-relevant primitive types — rather than a generic, unconstrained type system.
- A small, targeted set of optimizations (constant folding, dead/unreachable code elimination) chosen because they are both classically standard and concretely applicable to the condition-heavy structure of workflow programs, rather than an arbitrary or exaggerated optimization suite.

These elements are presented as realistic, scoped contributions suitable for individual implementation within the course timeline, not as claims of novel research or production-grade innovation.

17. Expected Outcomes

17.1 Outcomes of Phase 1 (Completed in This Document)

- A finalized problem statement, motivation, and objective set for the CodeFlow project.
- A formal, internally consistent CodeFlow language specification, including a complete EBNF grammar and operator precedence table.
- A documented system architecture and module decomposition covering the full compiler pipeline.
- A justified technology stack selection.
- An initial working prototype covering lexical analysis and basic parsing/AST construction, with basic syntax-error detection.

17.2 Outcomes Expected by Project Completion (Phases 2–3)

- A complete lexer and parser producing a full AST for all grammar constructs defined in Section 9.12.
- A working semantic analyzer and symbol table performing scope and type checking.
- Generation of correct Three-Address Code for semantically valid CodeFlow programs.
- Application of the planned optimizations (constant folding, dead/unreachable code elimination) with before/after IR comparison.
- A working execution engine capable of running representative CodeFlow programs end-to-end and reporting their action/alert/log effects.
- A visualization interface integrating all pipeline stages for demonstration and evaluation.